Comment planifier

🎯 Objectif de la phase PLANIFIER

La phase PLANIFIER est le pont entre SENSE et FORGE : c'est ici que ce qui a été ÉLICITÉ et DÉFINI est organisé, séquencé, découpé, assigné et outillé pour que l'implémentation, l'intégration, la diffusion et la confirmation soient prévisibles, coordonnées et mesurables.

> Planifier, ce n'est pas seulement mettre des stories dans un sprint. C'est orchestrer de A à Z tout ce qui va être fait : scope, découpage, assemblage, intégrations, automatisation, notifications, livraison, publication, métriques, signaux et responsabilités.

Note terminologique. Cette page décrit PLANIFIER dans son contexte opérationnel. Pour les définitions de référence des termes utilisés ici — SDLC, PLANIFIER, release, roadmap, EPIC, FEATURE, story, backlog, DoR, DoD, spécification atomique, dépendance, risque, mitigation, contrat d'intégration, environnement, pipeline CI/CD, test, validation, métrique, signal, observabilité, rôle et RACI — voir le Lexique / Terminologies. Les contributions indiquées dans cette page sont propres à la phase PLANIFIER et ne remplacent pas la matrice RACI.

Le principe fondamental : si ce n'est pas planifié, ce sera fait dans l'urgence — ou ne sera pas fait du tout. Tout ce qui doit arriver après PLANIFIER doit être anticipé : quoi, pourquoi, quand, qui, comment, avec quels outils, quels risques, quelles preuves et quels signaux.

📊 Les deux niveaux de planification

La planification se fait à deux niveaux complémentaires :

Niveau	Horizon	But
Niveau 1 — Portefeuille / release réelle	Release mineure ou majeure, roadmap, trimestre, PI	Planifier le scope global, les jalons, les dépendances, les risques, les contrats d'intégration, la communication, les métriques et la capacité macro.

Niveau 2 — Développement / release patch ou release de montage	Sprint, itération, incrément technique, patch	Découper les features en stories prêtes, planifier l'exécution détaillée, les tests, les intégrations, l'automatisation, les environnements, la livraison et les signaux opérationnels.

Le niveau 1 donne le cadre de release réel. Le niveau 2 transforme ce cadre en travail exécutable et remonte les risques, métriques et signaux nécessaires pour ajuster le plan.

🏗️ Niveau 1 — Planification portefeuille / release réelle

La planification portefeuille ou release réelle sert à organiser la trajectoire produit. Elle répond à la question : **qu'est-ce qu'on veut livrer, pourquoi, dans quel ordre, avec quelles dépendances et quels engagements ?**

Ce qui doit être planifié au niveau 1

Dimension	Ce qu'il faut planifier
Objectifs de release	Objectifs produit, valeur attendue, alignement roadmap, résultats mesurables et critères de succès.
Scope	EPICs, FEATURES, spécifications incluses, exclues, reportées, dépendantes ou conditionnelles.
Jalons	Dates clés, séquence de livraison, points de contrôle, décisions Go / No-Go, fenêtres de publication.
Dépendances	Dépendances inter-produits, inter-équipes, fournisseurs, plateformes, données, sécurité, exploitation et support.
Contrats d'intégration	Interfaces, API, événements, formats, versions, responsabilités, dates de disponibilité et critères de validation.

Risques	Risques de capacité, complexité, dette technique, réglementation, architecture, calendrier, qualité et adoption.
Capacité	Équipes disponibles, compétences, contraintes, congés, charge parallèle, besoins de renfort.
Communication	Rituels de synchronisation, revues de release, escalades, parties prenantes, notifications et canaux.
Métriques et signaux	Indicateurs de progression, qualité, risque, adoption, stabilité, seuils d'alerte, propriétaires et mécanismes de collecte.

Livrables du niveau 1

- Plan de release validé et approuvé.
- Backlog de release priorisé avec EPICs et FEATURES estimés à haut niveau.
- Calendrier de jalons, dépendances et décisions.
- Registre des risques avec stratégies d'atténuation.
- Contrats d'intégration confirmés avec les équipes partenaires.
- Plan de communication et de synchronisation.
- Plan de métriques : indicateurs, seuils, outils, propriétaires et fréquence de revue.

⚙️ Niveau 2 — Planification développement / patch / montage

La planification développement transforme la release réelle en travail concret. Elle répond à la question : **comment allons-nous réaliser, tester, assembler, livrer et mesurer chaque morceau ?**

Ce qui doit être planifié au niveau 2

Dimension	Ce qu'il faut planifier
Découpage	FEATURES découpées en stories claires, cohérentes, petites et traçables aux spécifications atomiques.

Couverture des spécifications	Chaque story indique les spécifications atomiques couvertes et les spécifications IF… THEN… associées.
Estimation fine	Effort, complexité, incertitude, capacité réelle et faisabilité technique validés par l'équipe.
Priorités	Ordonnancement validé par le PO selon valeur, risque, dépendances et séquence d'intégration.
DoR	Definition of Ready satisfaite avant entrée en implémentation.
Pré-requis techniques	Environnements, accès, données, configurations, secrets, outils, pipelines et dépendances disponibles.
Tests	Stratégie de tests small, medium, large / E2E, automatisation, données, environnements et responsabilités.
Assemblage	Branches, merges, builds, artefacts, versions, compatibilités et intégration continue.
Livraison et publication	Packaging, déploiement, rollback, notes de livraison, communication, transfert support et critères Go / No-Go.
Métriques opérationnelles	Collecte des signaux, dashboards, alertes, qualimétrie, observabilité et remontée vers ÉLICITER.

🔬 Checklist exhaustive : tout planifier de A à Z

1. Scope et découpage

- Quelles fonctionnalités sont incluses, exclues, reportées ou conditionnelles ?
- Quelles spécifications atomiques sont couvertes par chaque story ?
- Quel est le découpage minimal permettant de livrer de la valeur sans perdre la cohérence d'ensemble ?
- Quels morceaux doivent être assemblés ensemble pour former une release utilisable ?
- Quels éléments doivent être livrés en patch, en montage intermédiaire ou en release réelle ?

2. Automatisation

- Quels tests seront automatisés dès le départ ?
- Quels pipelines CI/CD doivent être créés, modifiés ou validés ?
- Quels contrôles automatiques bloquent un merge ou une livraison ?
- Quels scripts, jobs, générateurs, validateurs ou outils IA seront utilisés ?
- Qui maintient l'automatisation et comment les échecs sont-ils traités ?

3. Notifications et communication

- Qui doit être notifié à chaque étape : démarrage, blocage, intégration, build, test, livraison, publication, incident ?
- Par quel canal : Jira, Slack, Teams, courriel, standup, revue de release, dashboard ?
- Quelle fréquence de synchronisation entre équipes ?
- Quels événements déclenchent une escalade ?
- Qui communique aux parties prenantes, au support et aux utilisateurs internes ?

4. Arrimage avec les équipes

- Quelles équipes sont impliquées : développement, QA, architecture, sécurité, infra, données, support, produit, exploitation ?
- Quelles dépendances existent entre équipes ?
- Quels engagements doivent être confirmés avant de commencer ?
- Quels contrats d'intégration doivent être validés, par qui et quand ?
- Quels points de synchronisation empêchent les surprises en fin de cycle ?

5. Intégrations

- Quelles API, événements, fichiers, bases de données, services ou plateformes sont touchés ?
- Quels environnements permettent de tester l'intégration ?
- Quels jeux de données sont nécessaires ?
- Quels contrats doivent rester rétrocompatibles ?
- Quels tests prouvent que l'intégration fonctionne ?

6. Tests, validation et preuves

- Quels tests small, medium et large / E2E sont requis ?
- Quelles spécifications IF… THEN… sont validées par quelles preuves ?
- Quels tests sont automatisés et lesquels restent manuels par exception justifiée ?
- Qui exécute les tests, quand et sur quel environnement ?
- Où sont conservés les résultats, rapports de couverture et preuves de conformité ?

7. Assemblage, livraison et publication

- Quelle stratégie de branchement, merge et versionnement est utilisée ?
- Quels artefacts sont produits : binaires, images, packages, scripts, migrations, documentation ?
- Comment la release de montage devient-elle une release réelle ?
- Quel est le plan de déploiement et de rollback ?
- Qui prépare les notes de livraison, la publication et les communications associées ?

8. Risques et mitigation

- Quels sont les risques techniques, fonctionnels, opérationnels, calendrier, qualité, sécurité et adoption ?
- Quelle probabilité et quel impact pour chaque risque ?
- Quelle mitigation est planifiée ?
- Quel seuil transforme un risque en blocage ou escalade ?
- Qui est propriétaire de chaque risque ?

9. Métriques, signaux et observabilité

- Quelles métriques seront recueillies : avancement, vélocité, couverture, défauts, stabilité, performance, incidents, adoption, satisfaction ?
- Quels outils collectent les données : CI/CD, SonarQube, Jira, observabilité, logs, dashboards ?
- Quels seuils déclenchent une alerte ?
- Qui analyse les signaux et à quelle fréquence ?
- Comment les signaux remontent-ils vers ÉLICITER pour alimenter l'amélioration continue ?

10. Documentation et transfert

- Quelle documentation technique doit être créée ou mise à jour ?
- Quelle documentation utilisateur ou support est nécessaire ?
- Quels changements doivent apparaître dans le changelog ou les notes de livraison ?
- Quel transfert est requis vers CARE / support / exploitation ?
- Quels risques, limites, workarounds et parties non testées doivent être explicitement documentés ?

📌 Cas réel — Checklist PLANIFIER appliquée : Abandon simulation (RTLABTT-6023)

Cette section illustre comment instancier la checklist PLANIFIER sur une vraie feature. Elle peut servir de guide détaillé dans Confluence, tandis que Jira peut contenir une checklist haut niveau

dans le ticket pour piloter l'exécution quotidienne.

> **Principe.** Pour chaque feature, la planification doit couvrir non seulement l'implémentation, mais aussi le découpage, les versions livrées, les tests, les intégrations, la diffusion, la confirmation, les risques et les responsabilités.

1. Scope, découpage et version livrée

- [ ] Confirmer le scope final de la feature « Abandon simulation » : bouton, drag-and-drop, confirmation, cleanup et notifications.
- [ ] Documenter ce qui est inclus, exclu, conditionnel ou reporté.
- [ ] Valider la version cible de livraison et identifier ce qui doit être livré en release principale, patch ou montage intermédiaire.
- [ ] Vérifier que l'EPIC parent et les FEATUREs associées reflètent correctement le scope.
- [ ] Découper la feature en stories petites, traçables et livrables.
- [ ] Lier chaque story aux spécifications atomiques IF... THEN... correspondantes.

2. Planifier l'implémentation : qui fait quoi et quand

- [ ] Assigner les stories d'implémentation aux développeurs responsables.
- [ ] Définir la séquence d'implémentation : logique backend, UI, confirmation, drag-and-drop, cleanup, notifications.
- [ ] Identifier les reviewers et planifier les revues de code.
- [ ] Confirmer la stratégie de branchement, de merge et d'intégration continue.
- [ ] Identifier les dépendances techniques à clarifier avant de commencer.

3. Jeux de données, environnements et prérequis de validation

- [ ] Réserver les environnements de test requis, incluant le hardware ou les targets nécessaires.
- [ ] Documenter les prérequis matériels et logiciels pour reproduire les scénarios.
- [ ] Préparer les projets, modèles, configurations et données nécessaires aux validations.
- [ ] Prévoir les scénarios de target déconnectée, target qui revient UP, target unreachable et multiples targets disponibles.
- [ ] S'assurer que les environnements sont disponibles aux moments prévus pour QA, IVV et pair testing.

4. Tests, spécifications et exécutions Xray

- [ ] Valider que le Test Set Xray couvre toutes les spécifications IF... THEN....
- [ ] Créer les Test Executions Xray dans Jira et y associer les tests requis.
- [ ] Définir les niveaux de tests : small, medium, large / E2E.
- [ ] Assigner les exécutions aux responsables : dev, QA, IVV ou pair testing.
- [ ] Définir les critères pass/fail et les preuves attendues pour chaque test.
- [ ] Planifier les tests de non-régression sur les workflows existants.

5. Pair testing et responsabilités de validation

- [ ] Planifier le pair testing DEV × QA pour les zones fonctionnelles et les cas limites.
- [ ] Planifier le pair testing DEV × IVV pour l'intégration sur hardware réel et les scénarios système.
- [ ] Planifier le pair testing QA × IVV pour la validation E2E et la non-régression.
- [ ] Assigner clairement qui exécute, qui observe, qui documente et qui tranche les résultats.

6. Intégrations à planifier

- [ ] Identifier les interfaces impactées : API, state machine, UI, gestion réseau, target detection, drag-and-drop.

- [ ] Documenter les contrats d’intégration et les transitions d’état autorisées.
- [ ] Planifier les validations d’intégration avec les composants connexes.
- [ ] Confirmer la rétrocompatibilité des workflows existants : connect, run, stop, reset, cleanup.
- [ ] Identifier les features parallèles pouvant toucher les mêmes composants.

7. Diffuser : livraison, communication et support

- [ ] Planifier l’assemblage dans le build candidat et confirmer la branche cible.
- [ ] Planifier les tests de packaging, installation et smoke tests.
- [ ] Préparer les notes de livraison, limitations connues et workarounds.
- [ ] Planifier le transfert vers CARE / support : documentation, FAQ, dépannage, escalade.
- [ ] Définir les critères Go / No-Go pour la diffusion.

8. Confirmer : signaux après diffusion

- [ ] Définir les signaux à observer après diffusion : incidents, régressions, stabilité, feedback utilisateur.
- [ ] Identifier les métriques à recueillir : taux de succès, défauts, tickets support, stabilité des transitions d’état.
- [ ] Planifier les canaux de collecte : Jira, support, feedback terrain, dashboards ou logs.
- [ ] Définir la période d’observation post-diffusion et les responsables du suivi.

9. Risques et mitigation

- [ ] Identifier les risques calendrier, techniques, intégration, UX, hardware, qualité et adoption.
- [ ] Documenter pour chaque risque : probabilité, impact, propriétaire, mitigation et seuil d’escalade.
- [ ] Prévoir un scope minimal viable si la feature complète ne peut pas être livrée dans la version cible.
- [ ] Réserver tôt les ressources critiques, notamment les environnements hardware.
- [ ] Mettre à jour le registre des risques pendant toute la planification.

10. Checklist haut niveau suggérée pour Jira

Dans Jira, garder une checklist courte et actionnable, avec un lien vers cette section Confluence pour le détail.

- [ ] Scope et découpage validés.
- [ ] Version cible confirmée.
- [ ] Stories créées, estimées, priorisées et liées aux spécifications.
- [ ] Assignation et séquence/arrimage d’implémentation définies.
- [ ] Jeux de données/mocks, environnements et prérequis prêts
- [ ] Test Set et Test Executions Xray créés et assignés.
- [ ] Pair testing DEV × QA, DEV × IVV et QA × IVV planifié.
- [ ] Intégrations et contrats documentés.
- [ ] Plan de diffusion et transfert support préparés.
- [ ] Plan de confirmation et signaux post-diffusion définis.
- [ ] Risques documentés avec mitigation et propriétaires.
- [ ] DoR satisfait (critère d'entrée), DoD et DoT définis et planifiés (critères de sortie prêts à être vérifiés) avant entrée en IMPLÉMENTER

🔗 Arrimage entre les deux niveaux

Du niveau 1 vers le niveau 2	Du niveau 2 vers le niveau 1
Priorités, EPICs, FEATURES et jalons.	Capacité réelle, vélocité, complexité et risques découverts.
Contraintes de release, dates et fenêtres de publication.	Dépendances nouvelles ou non résolues.
Contrats d'intégration et engagements inter-équipes.	État des intégrations, blocages et besoins d'arbitrage.
Objectifs de qualité et métriques attendues.	Métriques réelles, tendances, alertes et signaux faibles.
Scope validé et exclusions connues.	Demandes d'ajustement du scope selon le terrain.

👥 Qui fait quoi pendant PLANIFIER ?

Rôle	Contribution
PM / responsable produit	Définit les objectifs de release, l'alignement roadmap, le scope macro et les arbitrages de valeur.
PO	Priorise les stories, valide le DoR, clarifie le besoin et arbitre le scope opérationnel.
Développeur	Estime, identifie les risques techniques, planifie le découpage, l'intégration, les tests unitaires et l'automatisation.
QA / validation	Planifie la stratégie de test, les niveaux de validation, les données, les environnements et les preuves.
Architecte	Valide les impacts systémiques, les contrats d'intégration, les choix techniques et la cohérence d'assemblage.

CdL	Coordonne le flux, suit la conformité SDLC, prépare les décisions de livraison et les communications de release.
Support / CARE	Planifie le transfert, la documentation support, les risques opérationnels et les besoins de formation.
VSE	Valide les arbitrages majeurs, le plan de release et les décisions critiques du flux de valeur.

✅ Definition of Ready — critère d'entrée en implémentation

Aucune story ne doit entrer en IMPLÉMENTER si elle n'est pas prête.

Critère DoR	Vérification
Clarté	La story est comprise par l'équipe, sans ambiguïté majeure.
Traçabilité	Les spécifications atomiques couvertes sont identifiées.
Testabilité	Les spécifications IF… THEN… sont mesurables et vérifiables.
Dépendances	Les dépendances sont résolues ou planifiées sans blocage.
Estimation	L'effort, la complexité et la capacité sont validés.
Pré-requis	Accès, environnements, données, outils, configurations et pipelines sont prêts.
Stratégie de test	Les tests requis, leur niveau, leur automatisation et leurs responsables sont définis.
Risques	Les risques sont identifiés, documentés et associés à une mitigation.

> Si le DoR n'est pas satisfait, la story retourne au backlog. On ne compense pas une mauvaise planification par de l'urgence en implémentation.

🔁 Boucles de rétroaction pendant PLANIFIER

Signal	Action
Spécification ambiguë ou incomplète	Retour à DÉFINIR avant de découper ou d'estimer.
Besoin non couvert découvert pendant le découpage	Remonter à ÉLICITER comme signal de besoin ou opportunité.
Dépendance non résolue	Escalader au niveau 1 et ajuster le plan ou le scope.
Capacité insuffisante	Réviser les priorités, réduire le scope ou ajuster les jalons.
Métriques historiques défavorables	Ajuster la capacité, la stratégie de test, l'automatisation ou les seuils.
Observabilité absente	Planifier sa mise en place comme prérequis, pas comme option.

⚠️ Pièges courants à éviter

Piège	Pourquoi c'est un problème	Comment l'éviter
Planifier seulement le code	Tests, intégration, livraison, documentation et métriques sont improvisés.	Utiliser la checklist exhaustive.
Oublier l'automatisation	Elle est repoussée et devient de la dette.	La planifier comme un livrable à part entière.
Ne pas planifier les notifications	Les équipes découvrent les blocages trop tard.	Définir les canaux, déclencheurs et responsables.
Découper sans traçabilité	Les stories perdent le lien avec les spécifications.	Lier chaque story aux spécifications atomiques.

Livraison planifiée à la fin	Packaging, rollback et publication sont improvisés.	Planifier la livraison dès le début.
Métriques absentes	On ne sait pas mesurer l'avancement, la qualité ou le succès.	Définir indicateurs, seuils, outils et propriétaires.

✅ Critères de sortie de PLANIFIER

Niveau 1 — Portefeuille / release réelle

- Les objectifs de release sont définis, mesurables et alignés avec la roadmap.
- Les EPICs et FEATURES sont priorisés, estimés à haut niveau et associés à des jalons.
- Le scope inclus, exclu, reporté ou conditionnel est explicite.
- Les dépendances inter-équipes et inter-produits sont identifiées et synchronisées.
- Les risques majeurs sont documentés avec mitigation et propriétaire.
- Les contrats d'intégration sont confirmés.
- Le plan de communication et de suivi est défini.
- Le plan de métriques, signaux et observabilité est défini.
- Le plan de release est validé par les parties prenantes clés.

Niveau 2 — Développement / patch / montage

- Les FEATURES sont découpées en stories claires, cohérentes et traçables.
- Chaque story couvre explicitement des spécifications atomiques.
- Les stories sont estimées finement et priorisées.
- Le backlog d'itération est ordonné et prêt.
- Les dépendances techniques et inter-équipes sont résolues ou planifiées.
- Les environnements, accès, données, outils et pipelines sont prêts.
- La stratégie de test et d'automatisation est définie.
- L'assemblage, la livraison, la publication et le rollback sont planifiés.
- Les métriques et signaux opérationnels sont configurés ou planifiés.
- Le DoR est pleinement satisfait pour les stories sélectionnées.

📊 Métriques et signaux

Les métriques de PLANIFIER servent à vérifier que le passage de DÉFINIR vers IMPLÉMENTER est réellement maîtrisé : le scope est clair, les dépendances sont connues, les stories sont

prêtes, les risques sont suivis et les signaux permettent d'ajuster le plan avant que les problèmes ne se transforment en urgence.

Métrique / signal	Ce qu'elle révèle	Source
Taux de stories avec DoR satisfait	Maturité de la planification — un taux faible indique qu'on entre en implémentation sans préparation suffisante.	Jira / workflow
Nombre de dépendances non résolues à l'entrée du sprint	Risque de blocage en implémentation et besoin d'arbitrage avant engagement.	Backlog / planning
Taux de replanification en cours d'itération	Stabilité du plan — un taux élevé indique un découpage, une estimation ou une anticipation insuffisante.	Jira / rapports de sprint
Couverture des spécifications atomiques par les stories	Traçabilité scope → réalisation — zéro lien indique des stories orphelines ou une couverture produit non démontrable.	Xray / liens Jira
Nombre de risques identifiés vs risques matérialisés	Qualité de l'anticipation — des risques matérialisés non identifiés révèlent des angles morts de planification.	Registre des risques
Délai entre fin de DÉFINIR et début d'IMPLÉMENTER	Fluidité du flux — un délai long indique un goulot en planification ou des prérequis non prêts.	Jira / dates de transition

🔄 Lien avec les autres phases

DÉFINIR produit des spécifications atomiques approuvées. PLANIFIER les transforme en scope, séquence, stories, risques, intégrations, automatisations, métriques et engagements. IMPLÉMENTER exécute ensuite ce plan. DIFFUSER publie ce qui a été assemblé et validé. CONFIRMER mesure les résultats, puis les signaux retournent vers ÉLICITER pour améliorer le système.

```
DÉFINIR → spécifications approuvées → PLANIFIER → plan exécutable → IMPLÉMENTER
    ↑                                              ↓
    └──── risques, dépendances, capacité, signaux ─────┘
```

> On ne peut pas bien implémenter ce qu'on n'a pas bien planifié. Et on ne peut pas bien planifier ce qu'on n'a pas bien défini.

Références

- Cérémonie de planification
- Comment définir — phase précédente
- Comment implémenter — phase suivante
- (SDLC) Software Development Life Cycle
- Contrat qualité officiel OPAL-RT
- RACI — Rôles et responsabilités
- Qualimétrie officielle OPAL-RT
- Lexique / Terminologies
- SPÉCIFICATION
- Exemples concrets de combinaisons
- Les tests — niveau × interface × angle × type
- Recommandations — enjeux, constats ou lacunes à résoudre / solutionner